

AGENCYOS

Admin Approval & Policy Engine

Document 07 of the AgencyOS Master Documentation Set

PRICING • DISCOUNTS • OFFERS • PROPOSALS • PAYMENT EXCEPTIONS • REFUNDS •
CANCELLATION • DEPLOYMENT • HANDOVER

Purpose: Define the policy and approval engine that determines which AI actions are automatically allowed, which require Admin approval, and how every governed decision is validated, executed and audited.

1. Core Principle

AI autonomy must be policy-bound. The Policy Engine decides whether an action is `AUTO_APPROVE`, `ADMIN_APPROVAL_REQUIRED`, `BLOCKED`, or `ESCALATE`.

- Agents may execute routine actions only inside approved policy boundaries.
- The UI is not the policy engine; backend authorization is authoritative.
- An agent cannot approve its own out-of-policy action.
- Every approval decision must be linked to the exact request, artifact/version, amount and policy evaluation.
- Policies must be configurable without changing core application code.
- Policy changes are themselves controlled, versioned and audited.

2. Policy Decision Model

AGENT REQUEST → NORMALIZE → LOAD POLICY → VALIDATE CONDITIONS → CALCULATE RISK → DECIDE → APPROVAL/EXECUTION → VERIFY → AUDIT

Possible decisions

- `AUTO_APPROVE` — action is fully inside policy.
- `ADMIN_APPROVAL_REQUIRED` — Admin must approve before execution.
- `BLOCK` — action is never allowed under the current policy.
- `ESCALATE` — insufficient information, conflict or unusual risk.
- `EXPIRED` — prior approval no longer applies because its validity window/version changed.
- `CANCELLED` — request withdrawn before execution.

3. Policy Object

Every policy should contain

- Policy ID.
- Policy name.
- Description.
- Business domain.
- Applicable agent(s).
- Applicable action(s).
- Conditions.
- Thresholds.
- Allowed values/ranges.
- Approval requirement.
- Required Admin role.
- Effective date.
- Expiry date where applicable.
- Policy version.
- Priority.
- Conflict-resolution rule.
- Enabled/disabled state.
- Created/updated by.

- Audit history.

4. Policy Evaluation Rules

- Specific policies should take precedence over generic policies.
- Explicit BLOCK should override AUTO_APPROVE.
- High-risk classifications should force approval even if another rule would otherwise allow automation.
- If required data is missing, do not guess — ESCALATE.
- If two policies conflict, use deterministic priority and surface the conflict.
- Policy evaluation must be reproducible from stored policy version + input data.
- The evaluated policy version must be recorded with the decision.

5. Risk Levels

Level	Meaning	Typical action
GREEN	Routine, low-risk, inside policy	Auto-approve
YELLOW	Moderate business/client impact	Policy-dependent; may require approval
ORANGE	Material commercial/operational impact	Admin approval
RED	High-risk financial, security, production or irreversible action	Mandatory Admin approval
BLACK	Forbidden action	Block

6. Pricing Policy

- Configure service/project categories.
- Configure base price.
- Configure allowed price range.
- Configure complexity tiers.
- Configure minimum acceptable price.
- Configure maximum autonomous quote value.
- Configure currency/tax behavior.
- Configure required approval threshold.
- Configure client-specific pricing rules where permitted.
- Configure emergency/manual pricing override.

Example: if AI calculates a quote inside the approved range, it may send it automatically. If below the minimum or outside the approved structure, Admin approval is required.

7. Discount Policy

- Maximum automatic discount percentage.
- Maximum automatic discount amount.
- Allowed discount by service/category.
- Discount stacking rules.
- Client-specific approved discount.
- First-project/new-client discount.
- Trust-case discount.

- Maximum negotiation rounds before escalation.
- Discount expiry.
- Admin approval threshold.

A Sales Agent may negotiate within these boundaries, but cannot invent a discount because the client asked repeatedly.

8. Offers & Special Deals

- Offer catalog.
- Eligibility conditions.
- Start/end date.
- Maximum quantity/usage.
- Applicable service/project types.
- Discount/benefit.
- Payment requirement.
- Approval level.
- Client-specific eligibility where authorized.
- Offer version.
- Audit trail.

AI can select an existing approved offer. Creating a new offer or materially changing an offer requires Admin approval.

9. Proposal / Quotation Policy

- Required fields.
- Scope completeness requirement.
- Pricing source.
- Payment plan.
- Timeline assumptions.
- Tax/GST treatment.
- Validity period.
- Revision/version number.
- Approval requirement.
- Client-facing release rule.

No quotation should be released if required scope, pricing, payment terms or approval information is missing.

10. Trust & No-Advance Exception Policy

Trust issues are a sales workflow, not permission for Sales to bypass commercial controls.

- Define approved low-advance structures.
- Define approved prototype/UI validation offers.
- Define when no-advance start is allowed, if the business chooses to support it.
- Define maximum work that may be completed before payment.
- Define which deliverables may be shown before payment.
- Define Admin approval threshold.

- Record client reason/trust concern.
- Record exact exception terms.
- Prevent an agent from silently extending the exception beyond its approved scope.

11. Payment-Term Policy

- Default milestone percentages.
- Minimum advance.
- Allowed milestone combinations.
- Maximum payment deferral.
- Payment due dates.
- Late-payment behavior.
- Whether work pauses automatically after overdue payment.
- Whether maintenance requires advance payment.
- Whether special payment terms require Admin approval.

Example default: 30% advance → 20% UI/prototype → 30% development → 20% final/testing. This is configurable, not a universal hard-coded rule.

12. Payment Exception Policy

- Client requests lower advance.
- Client requests payment after milestone.
- Client requests extended due date.
- Client requests split payment.
- Client requests alternative account/method.
- Payment mismatch.
- Payment proof unclear.
- Partial payment.
- Duplicate payment.
- Overpayment.

Any financial exception must have a deterministic state and approval path. A WhatsApp message alone is not sufficient to rewrite financial state.

13. Payment Verification Policy

PAYMENT CLAIMED → PROOF CAPTURED → PENDING_VERIFICATION → ADMIN/FINANCE VERIFICATION → VERIFIED / REJECTED / NEEDS_MORE_INFO

- Expected amount must be compared with claimed amount.
- Invoice must match the project/milestone.
- Reference/UTR or other evidence should be stored where available.
- Admin verification must record who verified it.
- Only VERIFIED payment may unlock payment-gated workflow states.
- Rejected/mismatched payment must stop the dependent transition.

14. Refund Policy

Refund rules

- Define refundable/non-refundable milestone types.
- Define refund eligibility windows.
- Define maximum refund amount.
- Define partial-refund rules.
- Define cancellation-linked refund rules.
- Define deductions for completed/accepted work.
- Define required evidence.
- Define Admin approval requirement.
- High-value refunds should require elevated Admin approval.

Refund execution must be idempotent and transaction-safe. Repeated requests must not create duplicate refunds.

15. Cancellation Policy

- Who may request cancellation.
- When cancellation is allowed.
- Project-state restrictions.
- Payment/refund consequences.
- Work already delivered.
- Work in progress.
- Client-caused delay.
- Agency-caused delay.
- Required Admin approval.
- Required final communication.
- Data retention/handover behavior.

AI may classify and prepare cancellation requests, but the final financial/project consequence must follow policy and required human approval.

16. Scope Change Policy

- Compare request against latest approved scope.
- Classify: in-scope, small free change, paid change request, new project or prohibited request.
- Define maximum free-change threshold.
- Define when a revised quote is required.
- Define whether timeline changes require approval.
- Require client acceptance for paid scope change.
- Version scope after approved change.
- Link change request to the milestone/project.

Developers must not silently implement out-of-scope work simply because a client mentioned it in chat.

17. Production Deployment Policy

Production deployment is a RED/high-risk action unless the Admin explicitly configures a lower-risk automated pathway.

- Required QA status.
- Required test status.
- Required security checks.
- Required build/version.
- Required client acceptance where applicable.
- Required payment status where contract requires it.
- Required deployment checklist.
- Required Admin approval.
- Post-deployment smoke test.
- Rollback plan.
- Deployment evidence.
- Production deployment audit event.

18. Handover Policy

- Required final payment state.
- Required client acceptance.
- Required QA/production-ready state.
- Required documentation.
- Required final build/version.
- Required credential-transfer procedure.
- Required invoice/receipt state.
- Required support/warranty terms.
- Required Admin approval.

Handover cannot be marked complete merely because files exist. Every required gate must be satisfied.

19. Project Completion Policy

- All mandatory milestones complete.
- All required payments verified.
- Final acceptance recorded.
- Handover complete.
- No unresolved blocking defect.
- Completion summary generated.
- Project marked COMPLETED.
- Historical records retained.

Completion is a controlled state transition, not an agent's free-text declaration.

20. Maintenance Policy

- Maintenance packages.
- Pricing.
- Billing period.
- Advance-payment requirement.
- Included support hours/features.

- SLA/priority rules.
- Renewal window.
- Grace period.
- Expiry behavior.
- Suspension behavior.
- Admin exception rules.

Maintenance access should activate only after the configured payment/contract condition is satisfied.

21. Upsell Policy

- Approved service catalog.
- Eligibility criteria.
- Offer/discount limits.
- Maximum autonomous offer value.
- Client-contact frequency limits.
- Approval thresholds.
- Prohibited/sensitive sales situations.
- Conversion tracking.
- Repeat-project classification.

Upsell Agent may identify opportunities, but should not spam clients or create unapproved commercial promises.

22. AI Model/Provider Policy

- Approved AI providers.
- Approved models.
- Blocked providers/models.
- Allowed data classes per provider.
- Per-agent provider rules.
- Fallback rules.
- Maximum cost per task.
- Global budget.
- Minimum quality threshold.
- Emergency provider disable.
- Production model pinning.

The Policy Engine governs what the Orchestrator may select; it does not replace the Orchestrator's routing logic.

23. Agent Permission Policy

Examples

- Sales Agent: create quote, but cannot approve own out-of-policy discount.
- PM Agent: advance a project only when required payment/approval gates are satisfied.
- Finance Agent: create invoice, but cannot self-verify an exception payment if Admin verification is required.
- Developer Agent: modify development branch, but cannot deploy production without required authorization.
- QA Agent: mark tests passed, but cannot approve a financial handover.
- Upsell Agent: create opportunity, but cannot invent new pricing.

- Orchestrator: route tasks, but cannot override business policy.

24. Approval Request Object

Required fields

- approval_id.
- organization_id.
- requesting_agent.
- request_type.
- client_id.
- project_id.
- task_id where applicable.
- exact action requested.
- input values.
- amount/value.
- artifact/version.
- policy version evaluated.
- risk level.
- reason.
- evidence.
- expiration.
- required Admin role.
- status.
- created_at.
- decision_at.
- decided_by.
- decision comment.

25. Approval States

DRAFT → SUBMITTED → UNDER_REVIEW → APPROVED / REJECTED /
CHANGES_REQUESTED / EXPIRED / CANCELLED → EXECUTED → VERIFIED

- APPROVED means permission was granted.
- EXECUTED means the action was actually performed.
- VERIFIED means the resulting state was checked where verification is required.

Do not collapse these states into one boolean such as approved=true.

26. Approval Expiry

- Approvals may expire if the underlying quote, scope, artifact or policy changes.
- Financial approvals should have configurable validity.
- Deployment approvals should apply to the exact build/version.
- Quotation approvals should apply to the exact quotation version.
- If a material input changes, the system should invalidate or re-evaluate the approval.

Approval is permission for a specific action under specific conditions — not a permanent blanket permission.

27. Approval Through WhatsApp

- Generate structured request with short human-readable summary.
- Include approval ID/reference.
- Provide controlled Approve / Reject / Request Changes interaction where supported.
- Backend authenticates the Admin.
- Backend verifies that the request is still valid.
- Backend records decision.
- Agent receives confirmed decision.
- If the message is ambiguous, do not execute.
- For high-risk actions, require explicit confirmation and/or in-panel approval according to policy.

28. Policy Simulation / Dry Run

Admin should be able to test a policy before activating it.

- Enter example quote amount.
- Enter discount.
- Enter client/project category.
- Enter payment plan.
- Select action.
- Run policy evaluation.
- Show expected decision.
- Show which policy rules matched.
- Show required approval.
- Show conflicts/warnings.

This lets Admin safely understand a rule before it changes live agent behavior.

29. Policy Versioning

- Every policy change creates a new version.
- Old versions remain auditable.
- Existing approval decisions reference the policy version used.
- Future actions use the current effective version.
- Scheduled future policies may have an effective date.
- Rollback creates a new version rather than silently rewriting history.

30. Policy Conflict Handling

- Detect multiple matching rules.
- Use explicit priority.
- Block ambiguous high-risk actions rather than guessing.
- Show conflict to Admin.
- Record which rules were considered.

- Require policy correction before repeated ambiguous execution.

31. Deterministic vs AI Decision

AI may recommend a decision; deterministic policy evaluation makes the authorization decision.

- AI can classify a request.
- AI can extract amount/scope/context.
- Backend Policy Engine checks exact rules and thresholds.
- Backend decides whether approval is required.
- AI cannot override a backend BLOCK.
- Admin approval cannot be faked by a model-generated sentence.

32. Override Policy

- Admin can override supported policies only through authorized controls.
- Override requires reason.
- Override requires identity.
- Override records before/after state.
- High-risk overrides require elevated role/confirmation.
- Temporary overrides should have expiry where appropriate.
- Every override becomes a permanent audit event.
- Automations must respect the override and not immediately undo it.

33. Emergency Policy Controls

- Disable all autonomous discounts.
- Disable new quotes.
- Disable payment exceptions.
- Disable refunds.
- Disable production deployment.
- Pause handovers.
- Pause outbound AI communication.
- Disable a provider/model.
- Switch agents to manual approval mode.
- Switch AgencyOS to degraded/manual operation.

Emergency controls must be simple, fast and backend-enforced.

34. Audit Requirements

- Record requesting agent/human.
- Record action.
- Record policy version.
- Record matched rules.
- Record decision.
- Record Admin identity.
- Record reason/comment.

- Record evidence.
- Record before/after state.
- Record execution result.
- Record verification result.
- Record timestamps.
- Protect audit history from silent modification.

35. Policy Security

- Only authorized Admin roles can modify policies.
- Policy secrets/credentials must never be stored in plain text.
- Policy endpoints require backend authorization.
- Sensitive policy changes may require re-authentication.
- Policy changes should generate security alerts where appropriate.
- No client-facing agent can modify policy.
- Agents read applicable policies; they do not rewrite them.

36. Policy Monitoring

- Approval rate.
- Auto-approval rate.
- Policy rejection rate.
- Escalation rate.
- Override rate.
- Most frequently triggered rules.
- Most common conflicts.
- Most common out-of-policy requests.
- Average approval time.
- Revenue affected by discounts/offers.
- Refund rate.
- Cancellation rate.
- Deployment approval rate.
- Handover exceptions.

These metrics help Admin improve policy quality instead of repeatedly solving the same exceptions manually.

37. Example Decision Flows

Example A — Standard quotation

Quote = inside approved price range + discount within limit + approved payment plan →
 AUTO_APPROVE → send to client.

Example B — Large discount

Discount exceeds autonomous threshold → ADMIN_APPROVAL_REQUIRED → Admin
 approves/rejects → if approved, send exact approved quote.

Example C — Payment exception

Client requests reduced advance → policy exception → ADMIN_APPROVAL_REQUIRED → approval references exact payment plan → Finance creates invoice.

Example D — Production deployment

QA passed + production candidate verified + required acceptance/payment conditions satisfied → ADMIN_APPROVAL_REQUIRED → deploy exact version → smoke test → record result.

Example E — Refund

Refund request → policy eligibility check → calculate allowed amount → elevated Admin approval if threshold exceeded → execute once → verify transaction → audit.

38. Acceptance Criteria

- Pricing rules are configurable.
- Discount rules are configurable.
- Offers are configurable.
- Quotation approvals are configurable.
- Payment exceptions require governed approval.
- Refunds have explicit policy and approval.
- Cancellation has explicit policy and financial consequence.
- Scope changes are policy-classified.
- Production deployment is approval-gated.
- Handover is approval-gated.
- Maintenance and upsell policies are configurable.
- AI provider/model policies are configurable.
- Agent permissions are policy-controlled.
- Approval requests are versioned and auditable.
- Approvals can expire when underlying conditions change.
- Policy simulation/dry-run is available.
- Policy conflicts are detected.
- Admin overrides are controlled and audited.
- Emergency controls exist.
- Backend enforcement prevents agents or UI from bypassing policy.

39. Final Policy Engine Architecture

AGENT/ADMIN REQUEST → POLICY ENGINE → RISK CLASSIFICATION → RULE MATCH → DECISION → APPROVAL (IF REQUIRED) → AUTHORIZED EXECUTION → VERIFICATION → AUDIT

The Policy Engine is the boundary between AI autonomy and business authority. It allows AgencyOS to move fast without allowing AI to make uncontrolled decisions about money, client commitments, production systems, refunds, cancellation or handover.

The goal is not to ask Admin for permission for everything. The goal is to make routine actions automatic and exceptional actions explicit, explainable, reversible where possible and fully auditable.